

OVERHEAD-AWARE ADAPTIVE OPENMP SCHEDULING

COURSE PROJECT REPORT

ON

COMPUTER ARCHITECTURE AND ORGANIZATION (CAO)

Submitted by: Sujitha S

Register Number: 26MAI0048

Project Guide :

Dr. Yokesh Babu S

Associate Professor Grade 1,

School of Computer Science and Engineering

Academic Year: 2026–2027

ABSTRACT

OpenMP loop-scheduling performance depends on workload distribution and the cost of runtime coordination. This project implements an overhead-aware adaptive scheduling prototype within a modified LB4OMP runtime. The runtime compares sampled STATIC and GUIDED execution times and switches policies only when projected savings over a reuse horizon exceed an estimated adaptation cost. Three deterministic workloads—uniform, irregular and increasing—were evaluated with STATIC, DYNAMIC, GUIDED and Adaptive scheduling. Five repetitions per configuration produced 60 benchmark executions, all of which passed correctness verification. GUIDED had the lowest median execution time across all three workloads. Relative to GUIDED, Adaptive was slower by 0.70%, 0.84% and 16.50%, respectively. For the increasing workload, Adaptive was approximately 31.0% faster than STATIC. The prototype demonstrates functioning cost-benefit scheduling decisions but also reveals the performance impact of exploratory rounds and incomplete overhead estimation. Reducing exploration cost and improving runtime decisions are identified as future work.

TABLE OF CONTENTS

CERTIFICATE

DECLARATION

ACKNOWLEDGEMENT

ABSTRACT

LIST OF FIGURES

LIST OF TABLES

1 INTRODUCTION

- 1.1 Project Overview
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope of the Project
- 1.5 Organization of Report

2 LITERATURE SURVEY

- 2.1 Existing Scheduling Systems
- 2.2 Limitations of Existing Systems
- 2.3 Proposed System

3 SYSTEM REQUIREMENTS

- 3.1 Functional Requirements
- 3.2 Non-Functional Requirements
- 3.3 Hardware and Software Requirements

4 SYSTEM DESIGN

- 4.1 System Architecture
- 4.2 Scheduling Decision Model

4.3 Execution Workflow

4.4 Workload and Benchmark Design

5 IMPLEMENTATION

5.1 LB4OMP Runtime Modification

5.2 Fixed Scheduling Policies

5.3 Overhead-Aware Adaptive Policy

5.4 Benchmark Implementation

5.5 Runtime Instrumentation

6 TESTING AND PERFORMANCE EVALUATION

6.1 Test Cases and Correctness

6.2 Experimental Setup

6.3 Performance Metrics

7 RESULTS AND DISCUSSION

7.1 Median Benchmark Results

7.2 Performance Comparison Graphs

7.3 Interpretation and Limitations

8 CONCLUSION AND FUTURE WORK

REFERENCES

APPENDICES

A. Source Code and Execution Commands

B. Benchmark Data

C. Runtime Logs and Repository Details

Note: Page numbers may be added or refreshed after final institutional formatting.

LIST OF FIGURES

Figure 4.1 Architecture of the overhead-aware adaptive OpenMP scheduling mechanism

Figure 7.1 Median execution time comparison across four scheduling policies

Figure 7.2 Adaptive scheduler compared with STATIC and GUIDED

LIST OF TABLES

Table 3.1 Hardware and software environment

Table 4.1 Benchmark workload characteristics

Table 6.1 Correctness and test coverage

Table 7.1 Median execution times

Table 7.2 Adaptive compared with best fixed scheduling policy

1 INTRODUCTION

1.1 Project Overview

OpenMP provides parallel loop scheduling through policies such as STATIC, DYNAMIC and GUIDED. Each policy makes different trade-offs between assignment overhead, locality and load balancing. This project investigates a modified runtime that uses measurements to decide whether a scheduling change is worthwhile.

1.2 Problem Statement

Runtime adaptation itself incurs costs. A faster candidate schedule may fail to compensate for the time spent sampling candidate policies, making decisions and applying a new policy. An adaptive mechanism should therefore consider both potential gains and adaptation costs.

1.3 Objectives

- Implement overhead-aware scheduling decisions within the LB4OMP runtime.
- Measure baseline and candidate scheduling performance during execution.
- Compare estimated savings against an adaptation-cost threshold.
- Verify correctness and compare performance with STATIC, DYNAMIC and GUIDED.
- Document the performance trade-offs and limitations of the prototype.

1.4 Scope of the Project

The implementation compares STATIC and GUIDED within the adaptive decision path; DYNAMIC is included as a fixed benchmark baseline. Evaluation covers three synthetic workloads, four threads, 17 rounds per execution and five repetitions. Machine-learning-based prediction is outside scope.

1.5 Organization of Report

Chapter 2 reviews scheduling approaches; Chapter 3 lists requirements; Chapter 4 explains system design; Chapter 5 describes implementation; Chapter 6 documents testing; Chapter 7 presents results; and Chapter 8 concludes with future work.

2 LITERATURE SURVEY

2.1 Existing Scheduling Systems

OpenMP STATIC scheduling distributes iterations with low coordination overhead but can perform poorly when iteration costs vary. DYNAMIC distributes chunks as threads become available and can improve load balancing at the cost of more runtime coordination. GUIDED gradually decreases chunk sizes and balances scheduling overhead against responsiveness to imbalance. LB4OMP provides a runtime framework for additional loop-scheduling algorithms and experimentation.

Prior adaptive approaches motivate selecting schedules based on observed workload behavior. The distinctive emphasis of this course project is to require a measured benefit estimate to exceed an explicit adaptation-cost estimate before switching.

2.2 Limitations of Existing Systems

A single fixed policy cannot guarantee the best performance for all workloads. Conversely, a naïve adaptive policy may incur exploration and switching costs that exceed its benefits. The ability to detect a faster candidate is therefore insufficient by itself to establish an end-to-end performance improvement.

2.3 Proposed System

The proposed system extends LB4OMP with a lightweight decision rule: sample STATIC and GUIDED, estimate savings over a reuse horizon, and retain STATIC unless expected savings exceed an estimated cost threshold. The approach uses timing observations and a deterministic comparison, without machine learning.

3 SYSTEM REQUIREMENTS

3.1 Functional Requirements

- Support OpenMP parallel-for execution through a modified runtime.
- Collect timing observations for STATIC and GUIDED sampling rounds.
- Estimate expected savings using measured average loop times.
- Apply a selected scheduling policy and optionally log the decision.
- Execute deterministic workloads and verify output checksums.

3.2 Non-Functional Requirements

- Correctness: identical expected results across scheduling policies.
- Reproducibility: fixed workload definitions and documented runtime settings.
- Efficiency: low decision overhead and limited instrumentation cost.
- Portability goal: compatibility with OpenMP and LLVM libomp interfaces, subject to build environment.

3.3 Hardware and Software Requirements

Component	Specification
Operating system	Ubuntu 26.04 LTS
CPU	AMD Ryzen 5 7500F, 6 cores / 12 threads
Memory	16 GB RAM
Compiler	Clang 21.1.8; GCC/G++ 15.2.0
Runtime	Modified LB4OMP / LLVM libomp
Language	C++17 with OpenMP
Benchmark	100,000 iterations, 17 rounds, 4 threads, 5 repetitions

4 SYSTEM DESIGN

4.1 System Architecture

The benchmark selects a workload and scheduling mode. Adaptive execution enters the modified LB4OMP runtime, where exploratory timing measurements feed an overhead-aware decision gate. The runtime retains STATIC or switches to GUIDED and applies the choice to later rounds.

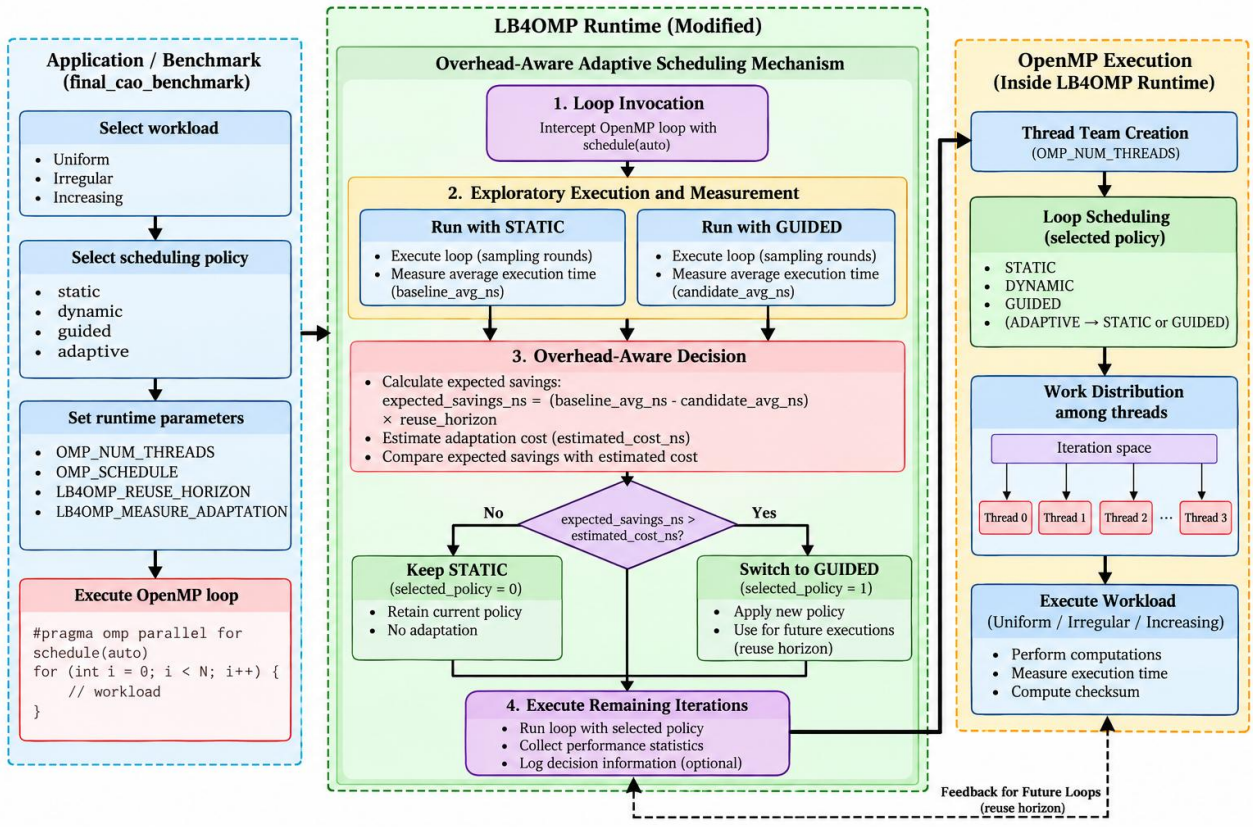


Figure 4.1. Architecture of the overhead-aware adaptive OpenMP scheduling mechanism.

4.2 Scheduling Decision Model

Expected savings (ns) = max(0, baseline_avg_ns − candidate_avg_ns) × reuse_horizon

Switch to GUIDED if expected_savings_ns > estimated_cost_ns;

otherwise retain STATIC.

The reuse horizon was set to 10, and the estimated cost threshold was approximately 1,766.2 ns in the tested implementation. This threshold was derived from limited runtime measurements and does not fully account for exploratory execution costs.

4.3 Execution Workflow

For the evaluated adaptive execution, round 1 serves as STATIC warm-up; rounds 2–4 establish the STATIC baseline; rounds 5–7 sample GUIDED; and the decision is made at round 8. The selected policy is used for subsequent rounds. Optional runtime logging records average times, estimated savings, cost and policy selection.

4.4 Workload and Benchmark Design

Workload	Purpose
Uniform	Approximately consistent iteration work.
Irregular	Variable iteration costs.
Increasing	Iteration cost increases across the iteration range.

Each workload is executed using STATIC, DYNAMIC, GUIDED and Adaptive scheduling. Fixed DYNAMIC and GUIDED use chunk size 16 in the final benchmark. The adaptive candidate configuration may differ internally, which limits strict policy-to-policy equivalence.

5 IMPLEMENTATION

5.1 LB4OMP Runtime Modification

The implementation uses a locally modified LB4OMP runtime, with scheduling logic integrated into the LLVM libomp dispatch path. The final benchmark links against the custom libomp shared library, not GCC libgomp.

5.2 Fixed Scheduling Policies

The benchmark runs explicit STATIC, DYNAMIC and GUIDED OpenMP schedules as comparison baselines. Their elapsed times include all 17 rounds of the final benchmark.

5.3 Overhead-Aware Adaptive Policy

Adaptive execution uses OpenMP runtime scheduling (OMP_SCHEDULE=auto,16), a reuse horizon of 10 and a cost-benefit comparison. When the measured GUIDED candidate is not sufficiently faster than STATIC, the runtime retains STATIC. A uniform-workload log recorded expected_savings_ns=0 and selected_policy=0, demonstrating rejection of an unprofitable switch in that run.

5.4 Benchmark Implementation

The final C++ benchmark accepts workload, policy and repetition parameters and produces CSV records containing workload, policy, repetition, iterations, rounds, elapsed time, checksum and correctness status. Each configuration executes 100,000 iterations for 17 rounds.

5.5 Runtime Instrumentation

Optional LB4OMP_MEASURE_ADAPTATION logging exposes decision details, including baseline and candidate averages, expected savings, estimated cost and selected policy. The reported cost estimate includes measured decision and application components but excludes some exploratory execution cost.

6 TESTING AND PERFORMANCE EVALUATION

6.1 Test Cases and Correctness

Test	Expected	Observed
Uniform workload across four policies	Matching checksum and PASS	20/20 PASS
Irregular workload across four policies	Matching checksum and PASS	20/20 PASS
Increasing workload across four policies	Matching checksum and PASS	20/20 PASS
Total	60 valid executions	60/60 PASS

Checksums were consistent across policies for each workload:

uniform 14146936862410912037;
irregular 8274984843985544357;
increasing 15841823406551480615.

6.2 Experimental Setup

The final study executed 3 workloads × 4 policies × 5 repetitions = 60 runs.

All runs used 100,000 iterations, 17 rounds and four OpenMP threads. Execution times were summarized using the median of five repetitions per workload-policy combination.

6.3 Performance Metrics

Primary metric: end-to-end execution time in milliseconds (lower is better).

Comparative improvement (%) = (reference_time − adaptive_time) / reference_time × 100.

Negative values indicate that Adaptive is slower than the reference.

7 RESULTS AND DISCUSSION

7.1 Median Benchmark Results

Workload	STATIC	DYNAMIC	GUIDED	Adaptive
Uniform	118.018	117.445	116.708	117.524
Irregular	118.585	118.467	117.394	118.378
Increasing	195.761	117.292	115.966	135.105

Table 7.1. Median execution times (ms), five repetitions per configuration. GUIDED achieved the lowest median time in each workload.

Workload	Best fixed	Best fixed (ms)	Adaptive (ms)	Improvement
Uniform	GUIDED	116.708	117.524	−0.70%
Irregular	GUIDED	117.394	118.378	−0.84%
Increasing	GUIDED	115.966	135.105	−16.50%

Table 7.2. Adaptive performance relative to the best measured fixed policy.

7.2 Performance Comparison Graphs

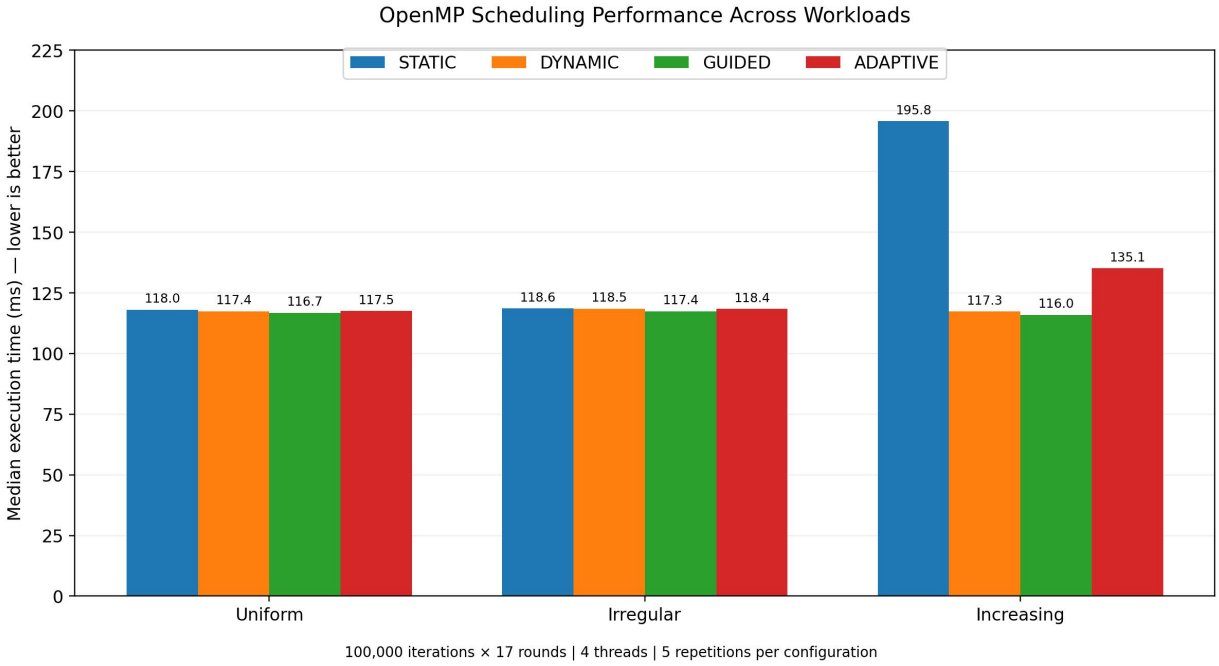


Figure 7.1. Median execution time comparison across scheduling policies.

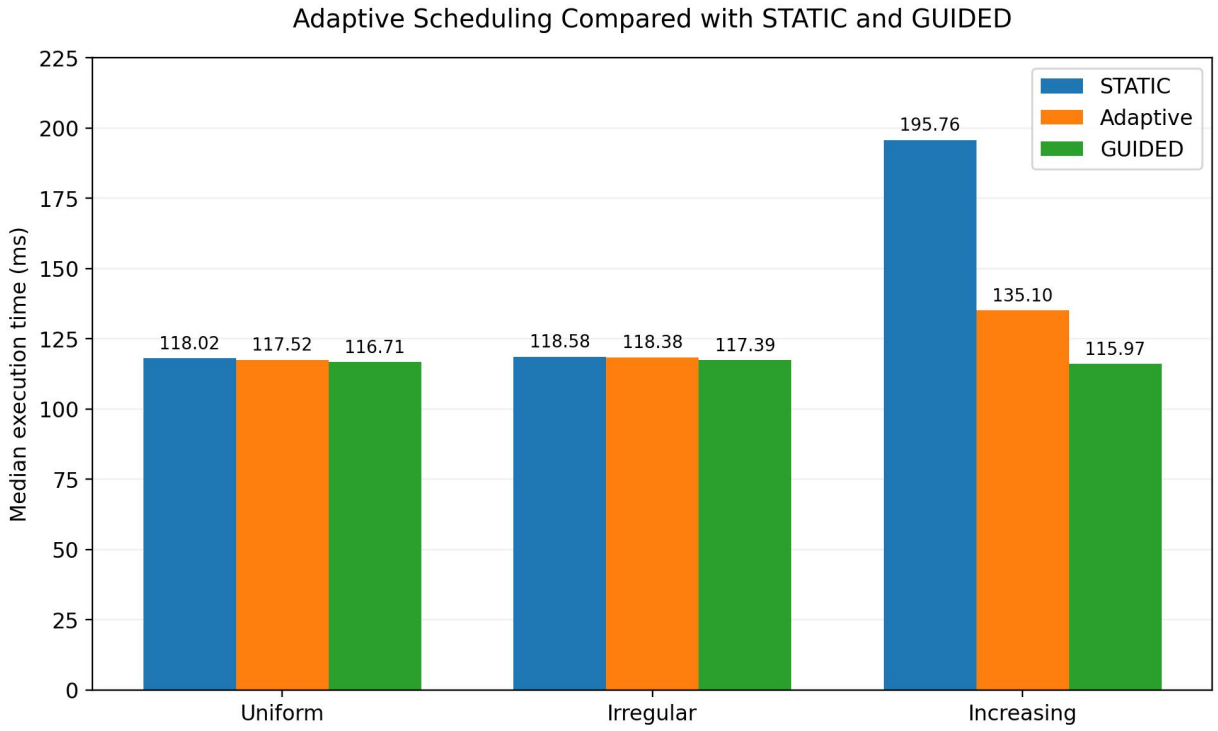


Figure 7.2. Adaptive scheduling compared with STATIC and GUIDED.

7.3 Interpretation and Limitations

For uniform and irregular workloads, Adaptive was within approximately 1% of GUIDED. For the increasing workload, Adaptive reduced time from 195.761 ms (STATIC) to 135.105 ms, an improvement of approximately 31.0% over STATIC. However, GUIDED completed the same workload in 115.966 ms, so Adaptive was 16.50% slower than the best fixed baseline.

The adaptive scheduler functions as designed at the decision level, but the current evaluation does not show superiority over the best fixed policy. Exploratory rounds incur execution cost, the estimated adaptation threshold omits some overhead components, and the comparison is limited to one machine, one thread count, three synthetic workloads and five repetitions. The fixed GUIDED chunk size also differs from the adaptive runtime candidate configuration in some cases.

8 CONCLUSION AND FUTURE WORK

The course project implemented an overhead-aware adaptive OpenMP scheduling prototype within LB4OMP and completed a correctness-verified comparative benchmark. The runtime measures scheduling behavior, estimates projected savings and retains or changes policy based on a cost threshold. All 60 final executions passed correctness checks. GUIDED achieved the best overall median times; Adaptive was faster than STATIC on the increasing workload but did not surpass GUIDED. The principal contribution is a working cost-benefit decision mechanism and an experimental analysis of its limitations.

Future work includes reducing sampling overhead, measuring all adaptation costs, adjusting the reuse horizon dynamically, considering additional candidate policies such as DYNAMIC and other LB4OMP algorithms, and evaluating real-world scientific, image-processing and graph workloads across multiple machines and thread counts.

REFERENCES

[1] OpenMP Architecture Review Board, OpenMP API Specification, https://www.openmp.org/specifications/.

[2] LB4OMP, load-balancing scheduling framework for OpenMP, project source repository and associated documentation (runtime foundation).

[3] Project-generated benchmark results and runtime decision logs, October 2026.

APPENDICES

A. Source Code and Execution Commands

Main project source: final_cao_benchmark.cpp.

Runtime modification: LB4OMP runtime/src/kmp_dispatch.cpp.

The source code itself is maintained in the local project directory and is not reproduced here because it was not supplied with the benchmark CSV.

Representative adaptive settings:

OMP_NUM_THREADS=4;

OMP_SCHEDULE=auto,16;

KMP_Golden_Chunksize=0;

LB4OMP_REUSE_HORIZON=10.

Optional logs: LB4OMP_MEASURE_ADAPTATION=1.

B. Benchmark Data

The complete 60-row CSV is provided as final_cao_results.csv alongside this report. Its columns are workload, policy, repetition, iterations, rounds, time_ms, checksum and correct.

C. Runtime Logs and Repository Details

The tested runtime library is the locally built LB4OMP libomp.so. Representative decision logs include baseline_avg_ns, candidate_avg_ns, expected_savings_ns, estimated_cost_ns and selected_policy. Full source diffs and runtime logs should be attached if required by the faculty.